

Universidad Mariano Gálvez
Facultad de Ingeniería en Sistemas de Información
Y Ciencias de la Computación
Curso: Desarrollo Web

Proyecto final de curso

🎮 Aprendo Fácil Jugando 🎮

Aplicación web educativa para preprimaria y primaria

INFORME DEL PROYECTO

Aplicación publicada: https://aprendo-facil-jugando.onrender.com
API publicada: https://aprendo-facil-api.onrender.com/api/actividades

El código completo se entrega en el archivo comprimido del proyecto

Estudiante: Luis Alberto López Hernández
Carné: 5990-18-1186
Catedrático: Edgar Jehomías Matul García

Guatemala, Octubre 2,026

1. Introducción

Aprendo Fácil Jugando es la aplicación web que desarrollé como proyecto final del curso de Desarrollo Web. La idea es sencilla: que niños de preprimaria y primaria refuercen lo que aprenden en clase jugando, y que sus docentes puedan ver cómo les va a cada uno sin tener que revisar nada a mano.

Para construirla usé el stack que pide el curso: React con CSS en el frontend, Node.js con Express en el backend y MySQL como base de datos. El frontend y el backend se comunican por medio de una API REST con CORS habilitado, de modo que cada resultado que genera un niño se guarda de forma permanente en la base de datos.

Primero la desarrollé y probé en mi computadora y después la publiqué en internet con Render (frontend y backend) y Aiven (MySQL), para que cualquier persona pueda abrirla desde un navegador. En este informe explico qué hice, cómo está organizado el proyecto y qué evidencia tengo de que cumple cada requerimiento.

2. Objetivos

2.1 Objetivo general

Desarrollar una aplicación web interactiva para la educación infantil, con una arquitectura full-stack (React, Express y MySQL), que ayude a los niños a reforzar su aprendizaje básico mediante juegos y que permita a docentes y tutores dar seguimiento a su rendimiento.

2.2 Objetivos específicos

- Diseñar e implementar un frontend interactivo con React y CSS, visualmente atractivo, accesible y pensado para niños.
- Construir una API REST con Express y CORS que comunique la interfaz con la lógica de negocio y la base de datos de forma segura e independiente.
- Modelar e integrar una base de datos relacional en MySQL que conserve estudiantes, actividades y el historial de progreso.

3. Descripción del sistema

La aplicación tiene dos tipos de usuario. El estudiante escribe su nombre y edad, elige su nivel y juega; el docente entra a un tablero donde ve el historial de partidas y puede filtrarlo por estudiante. Cada acierto vale 10 puntos y el resultado se guarda al terminar la partida.

Nivel	Actividad	Qué practica el niño	Preguntas por partida
Preprimaria	Adentro y Afuera	Ubicar objetos dentro o fuera de una caja	5
Preprimaria	Arriba y Abajo	Ubicación vertical de objetos	5
Preprimaria	Cerca y Lejos	Distancia entre dos elementos	5
Preprimaria	Encima y Debajo	Posición de un objeto respecto a otro	5

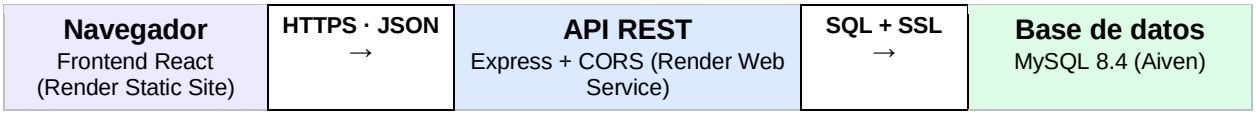
Nivel	Actividad	Qué practica el niño	Preguntas por partida
Preprimaria	Figuras Geométricas	Reconocer círculo, cuadrado, triángulo y rectángulo	8
Preprimaria	Memoria de Pares	Encontrar parejas de cartas iguales	6 parejas
Primaria	Tablas de Multiplicar	Multiplicaciones del 2 al 10	8
Primaria	Suma de Quebrados	Sumar fracciones con distinto denominador	8
Primaria	Multiplicación con Decimales	Multiplicar números decimales	8

4. Herramientas y tecnologías

Herramienta	Para qué la usé	Detalle
Visual Studio Code	Escribir el código del backend y el frontend	Terminal integrada de PowerShell
Node.js	Ejecutar el backend y las herramientas del frontend	v24 en mi equipo; v24 en Render
React + Vite	Construir la interfaz por componentes	Servidor de desarrollo en el puerto 5173
Express, cors, dotenv, mysql2	API REST, permisos entre orígenes, variables de entorno y conexión a MySQL	Puerto 10001 en local
MySQL 8 y MySQL Workbench	Base de datos local y administración	Versión 8.0.46 en mi equipo
Aiven (MySQL)	Base de datos en la nube	Plan gratuito, MySQL 8.4
Render	Publicar el backend (Web Service) y el frontend (Static Site)	Plan gratuito, región Virginia
Git y GitHub	Control de versiones y fuente de los despliegues	Repositorio privado
Navegador (DevTools)	Depurar errores y medir tiempos de respuesta	Pestañas Console y Network
Recursos gráficos	Imágenes educativas, fondo, emojis y tipografía Fredoka	Créditos al final del informe

5. Arquitectura

El proyecto está dividido en tres piezas independientes que solo se comunican por red. El navegador ejecuta React; cuando necesita datos o guardar algo, hace una petición HTTP a la API de Express; y esta consulta o escribe en MySQL.



Un ejemplo del recorrido: cuando un niño termina un juego, React llama a `guardarProgreso()`, que envía un POST a `/api/progreso`. Express valida los datos, ejecuta un INSERT en la tabla `progreso` y responde con un mensaje; entonces la pantalla muestra "Partida guardada".

Entorno	Frontend	Backend	Base de datos
Local (mi computadora)	localhost:5173	localhost:10001	MySQL local, puerto 3306
Nube (publicado)	Render Static Site	Render Web Service	Aiven MySQL



Figura 1. Diagrama de la estructura de carpetas del proyecto (backend y frontend) en VS Code

6. Base de datos

Diseñé tres tablas. `estudiantes` guarda al niño que se registra, `actividades` es el catálogo de juegos (el menú se arma a partir de ella) y `progreso` guarda cada partida y se relaciona con las otras dos mediante llaves foráneas, de modo que MySQL no deja guardar resultados de un estudiante o una actividad que no existen.

Tabla	Columnas	Función
estudiantes	id, nombre, edad, creado_en	Registro del niño (RF-01)
actividades	id, nombre, nivel, tema	Catálogo de juegos (RF-07)
progreso	id, estudiante_id, actividad_id, puntaje, aciertos, intentos, fecha	Historial de partidas, con dos FOREIGN KEY (RF-05, RNF-04)

Script completo en `database.sql`:

```
CREATE DATABASE IF NOT EXISTS aprendo_facil CHARACTER SET utf8mb4;
USE aprendo_facil;

CREATE TABLE IF NOT EXISTS estudiantes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(60) NOT NULL,
  edad TINYINT NOT NULL,
  creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS actividades (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(60) NOT NULL,
  nivel ENUM('preprimaria','primaria') NOT NULL,
  tema VARCHAR(40) NOT NULL
);

CREATE TABLE IF NOT EXISTS progreso (
  id INT AUTO_INCREMENT PRIMARY KEY,
  estudiante_id INT NOT NULL,
  actividad_id INT NOT NULL,
  puntaje INT NOT NULL,
  aciertos INT NOT NULL,
  intentos INT NOT NULL,
```

```
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (estudiante_id) REFERENCES estudiantes(id),
  FOREIGN KEY (actividad_id) REFERENCES actividades(id)
);

INSERT INTO actividades (nombre, nivel, tema) VALUES
('Adentro y Afuera','preprimaria','espaciales'),
('Arriba y Abajo','preprimaria','espaciales'),
('Cerca y Lejos','preprimaria','espaciales'),
('Encima y Debajo','preprimaria','espaciales'),
('Figuras Geométricas','preprimaria','figuras'),
('Memoria de Pares','preprimaria','memoria'),
('Tablas de Multiplicar','primaria','matematicas'),
('Suma de Quebrados','primaria','matematicas'),
('Multiplicación con Decimales','primaria','matematicas');
```

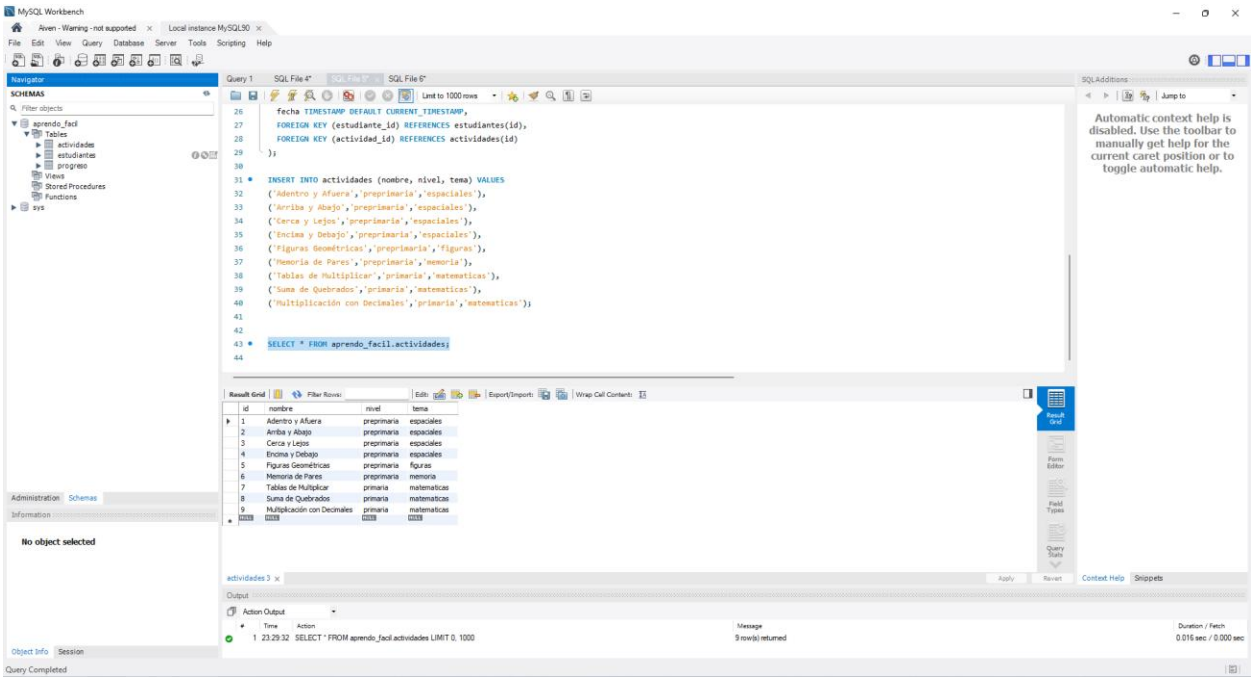


Figura 2. Creación BD en MySQL Workbench (conexión local)

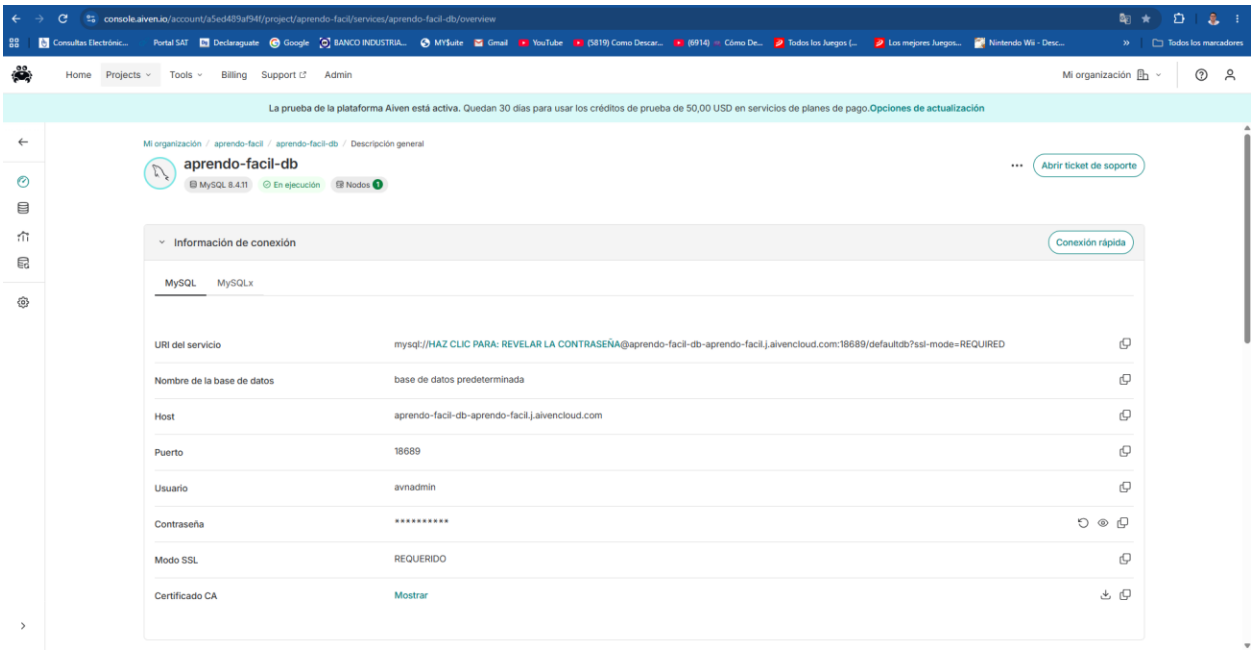


Figura 3. Servicio MySQL en Aiven

7. Instalación y ejecución local

Las instrucciones completas están en el archivo `README.md` del proyecto. En resumen, para probarlo en una computadora se necesita Node.js 18 o superior y MySQL 8.0 o superior:

- Ejecutar `database.sql` en MySQL para crear la base `aprendo_facil` con sus tablas y las 9 actividades.
- En la carpeta `backend`: `npm install`, copiar `.env.example` como `.env` (escribiendo la contraseña de MySQL) y ejecutar `node server.js`.
- En la carpeta `frontend`: `npm install`, copiar `.env.example` como `.env` y ejecutar `npm run dev`.
- Abrir `http://localhost:5173` en el navegador.

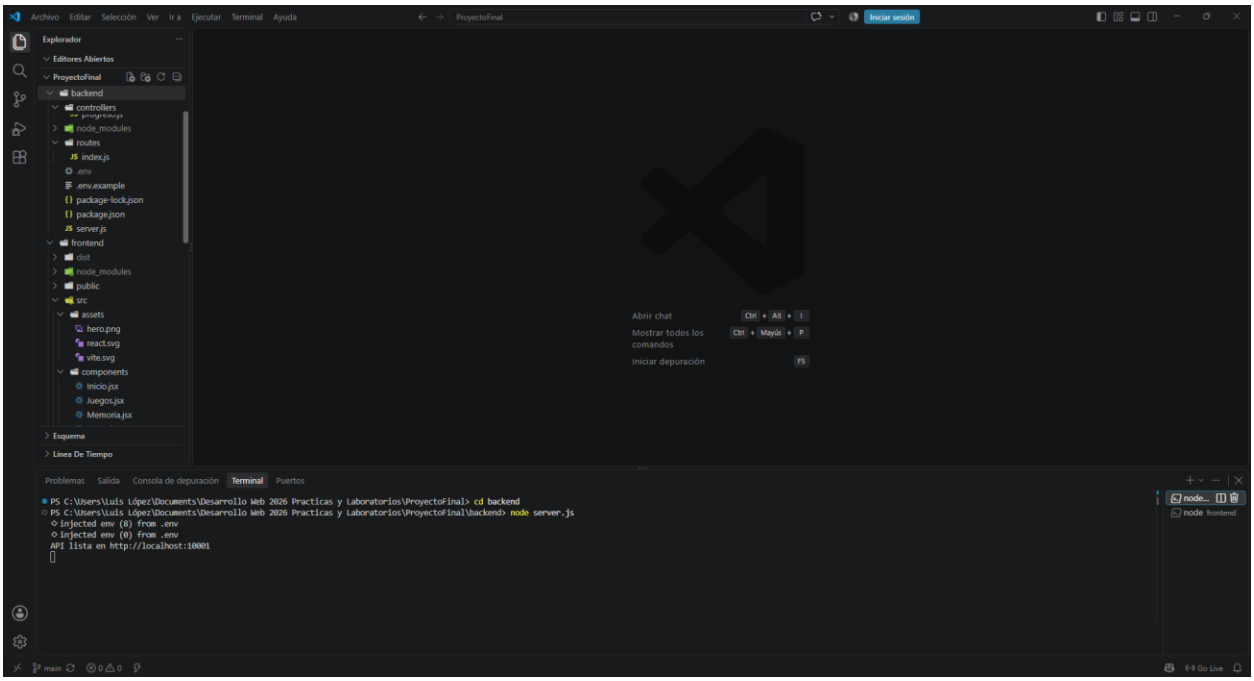


Figura 4. Terminal con el backend en ejecución node server.js = API lista en http://localhost:10001

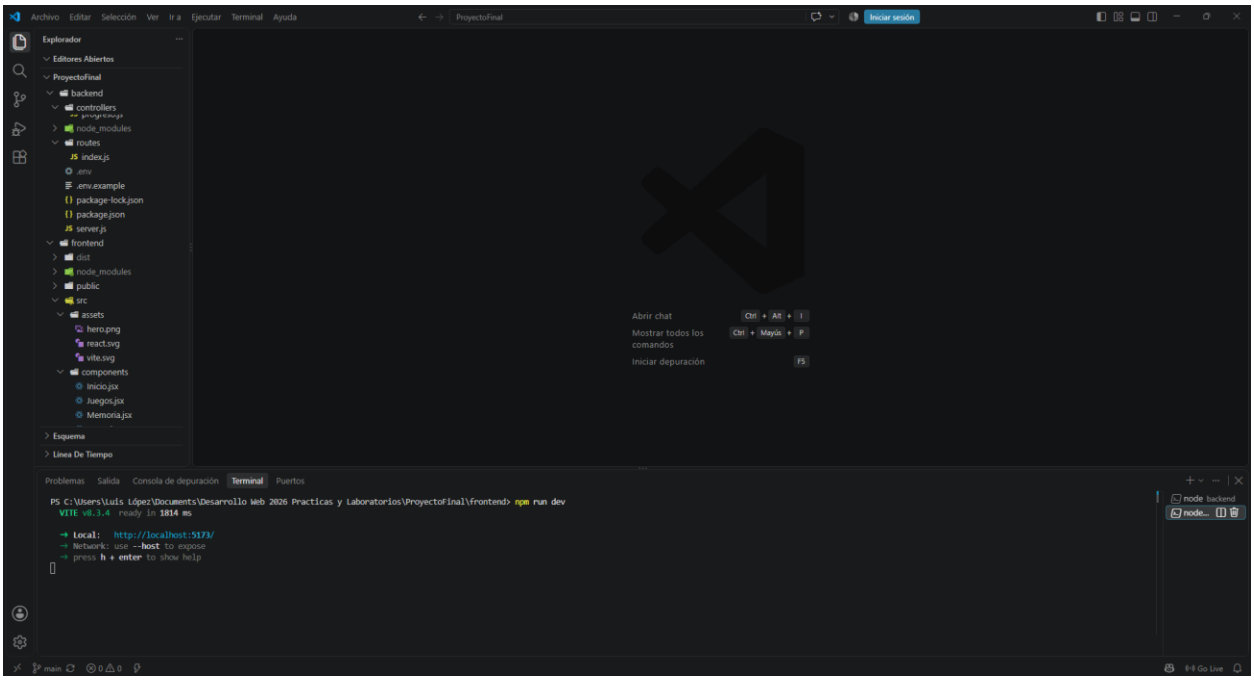


Figura 5. Terminal con el frontend en ejecución npm run dev,= Local: http://localhost:5173/

8. Desarrollo del backend

El backend lo organicé en tres capas: `server.js` arma la aplicación, `routes/index.js` define las direcciones y `controllers/` contiene la lógica de cada recurso. La conexión a MySQL está en `config/db.js` y usa un pool de conexiones. Todo dato sensible (host, usuario, contraseña) sale de variables de entorno y no del código.

Método	Ruta	Qué hace
POST	/api/estudiantes	Registra un estudiante (valida nombre y edad de 3 a 12)
GET	/api/estudiantes	Lista los estudiantes, ordenados por nombre
GET	/api/actividades?nivel=	Devuelve el catálogo; el filtro por nivel es opcional
POST	/api/progreso	Guarda el resultado de una partida (valida los datos)
GET	/api/progreso?estudiante_id=	Historial con nombres (JOIN); filtro opcional por estudiante

Archivo `server.js`, donde se activan CORS, la lectura de JSON y las rutas:

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');

const app = express();
app.use(cors({ origin: process.env.FRONTEND_URL }));
app.use(express.json());
app.get('/', (req, res) => {
  res.json({ nombre: 'API Aprendo Fácil Jugando', estado: 'en línea',
actividades: '/api/actividades' });
});
app.use('/api', require('./routes'));

app.use((err, req, res, next) => {
    console.error(err);
    res.status(500).json({ error: 'Error del servidor' });
});

app.listen(process.env.PORT, () =>
    console.log(`API lista en http://localhost:${process.env.PORT}`)
);
```

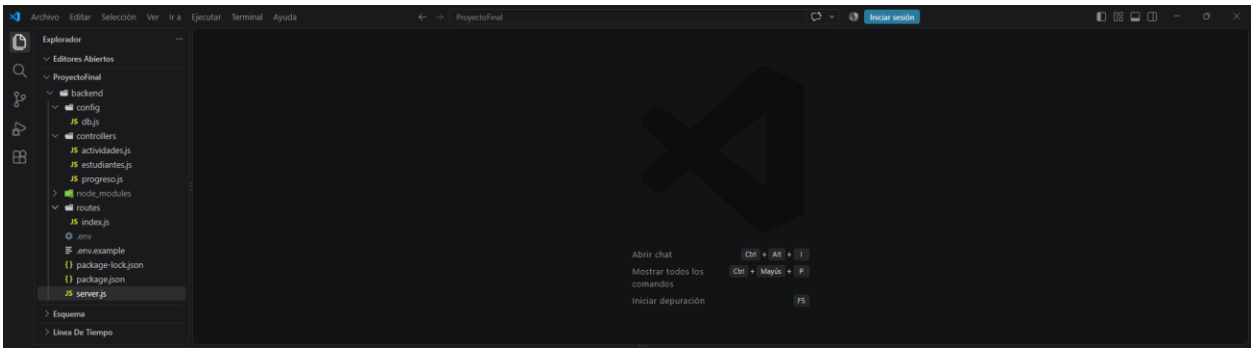


Figura 6. Carpeta backend abierta en VS Code mostrando config, controllers y routes

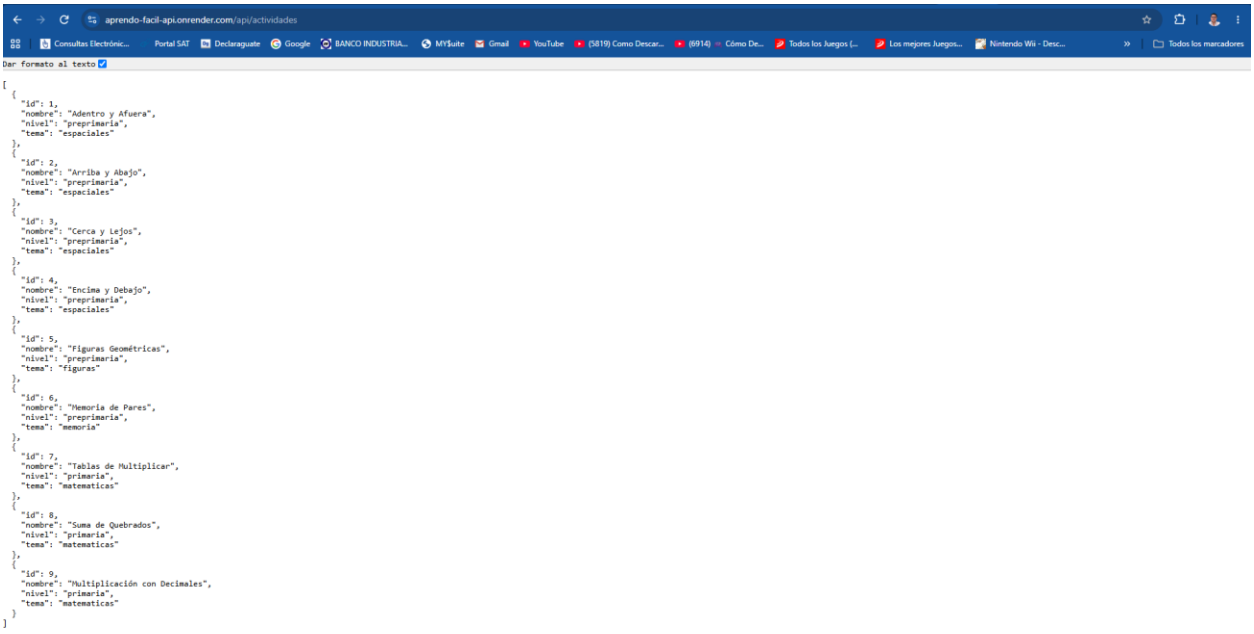


Figura 7. Respuesta JSON de /api/actividades en el navegador (versión publicada en Render)

9. Desarrollo del frontend

En el frontend dividí la interfaz en componentes: `Inicio`, `Registro`, `Menu`, `Juegos`, `Memoria` y `Tablero`. El archivo `App.jsx` decide cuál mostrar según tres estados (rol, estudiante y actividad), por lo que no necesité librerías de rutas. Todas las llamadas al backend están en `api.js`, y las preguntas de los juegos las genera `data/preguntas.js`.

Pantalla de inicio

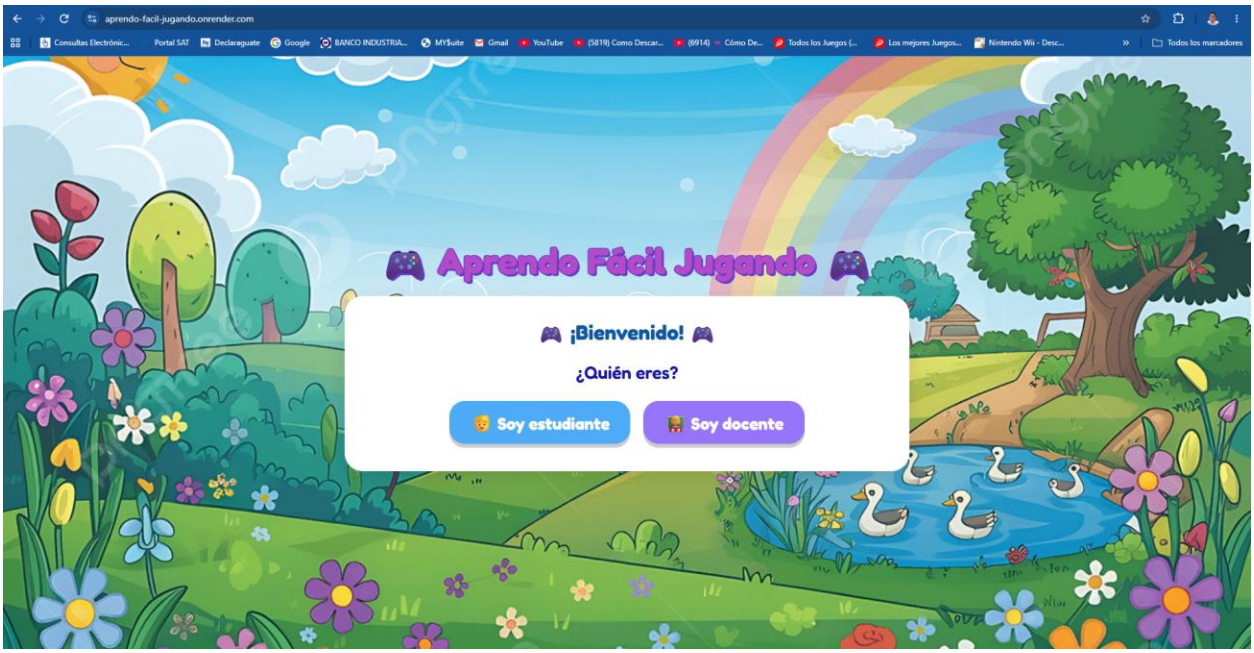


Figura 8. Pantalla de inicio con los botones "Soy estudiante" y "Soy docente"

9.1 Registro del estudiante

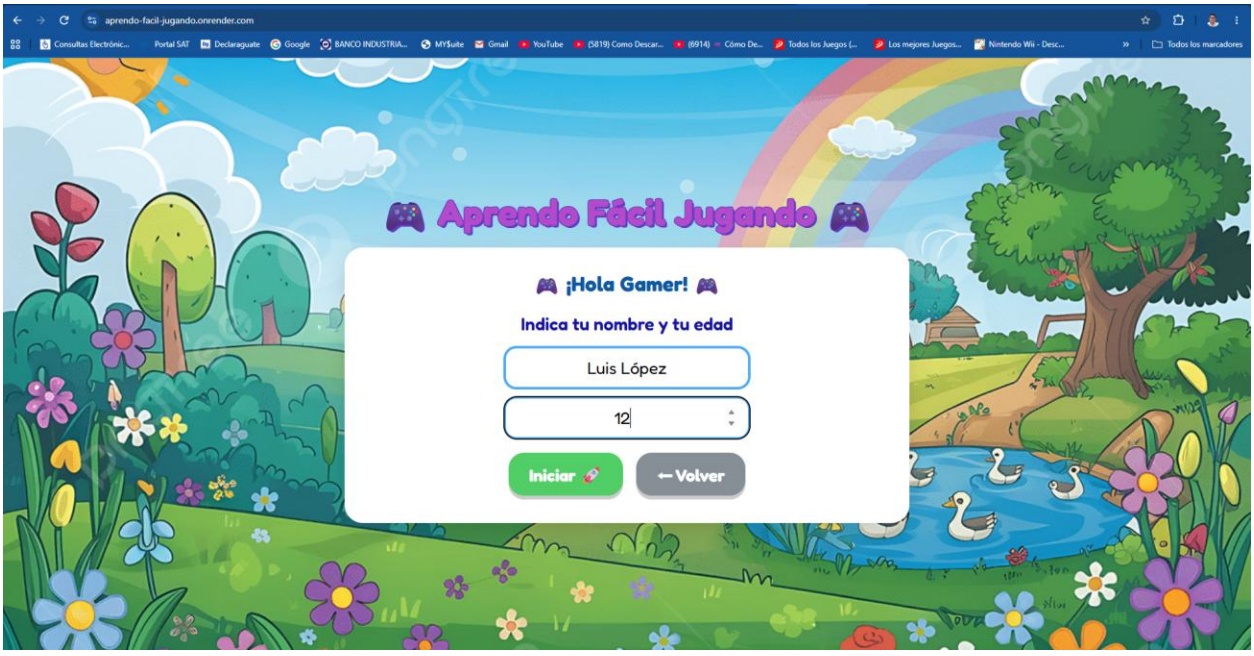


Figura 9. Pantalla de registro con nombre y edad

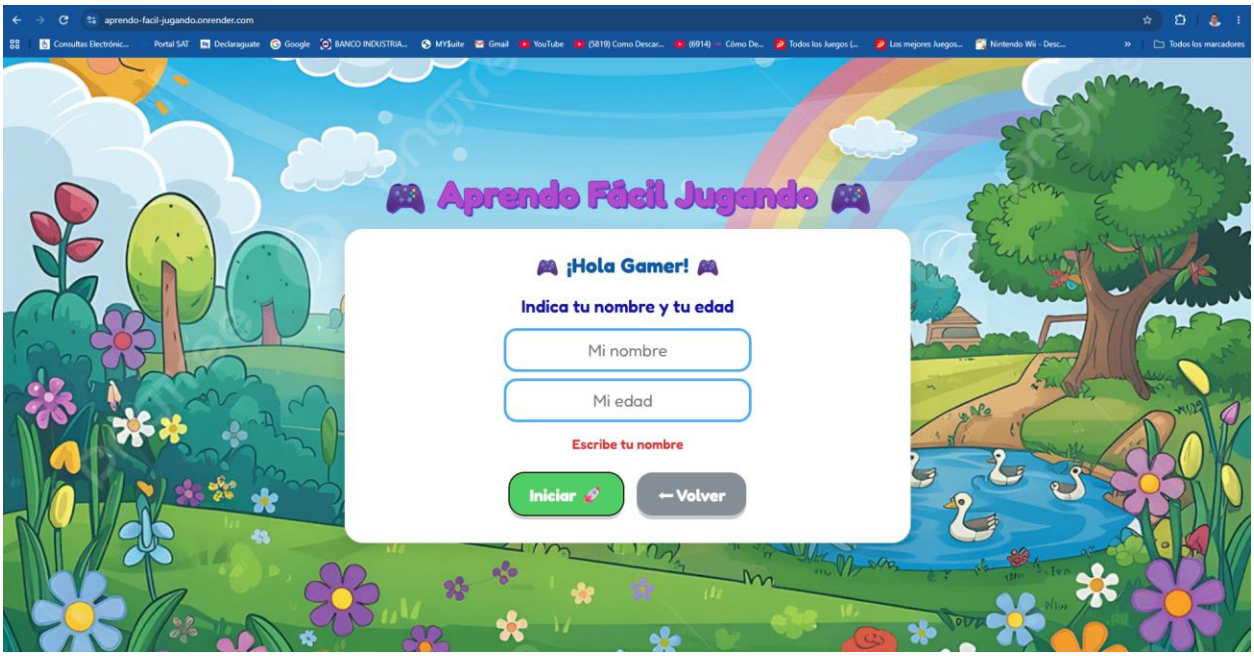


Figura 10. Registro con un campo vacío mostrando el mensaje de validación

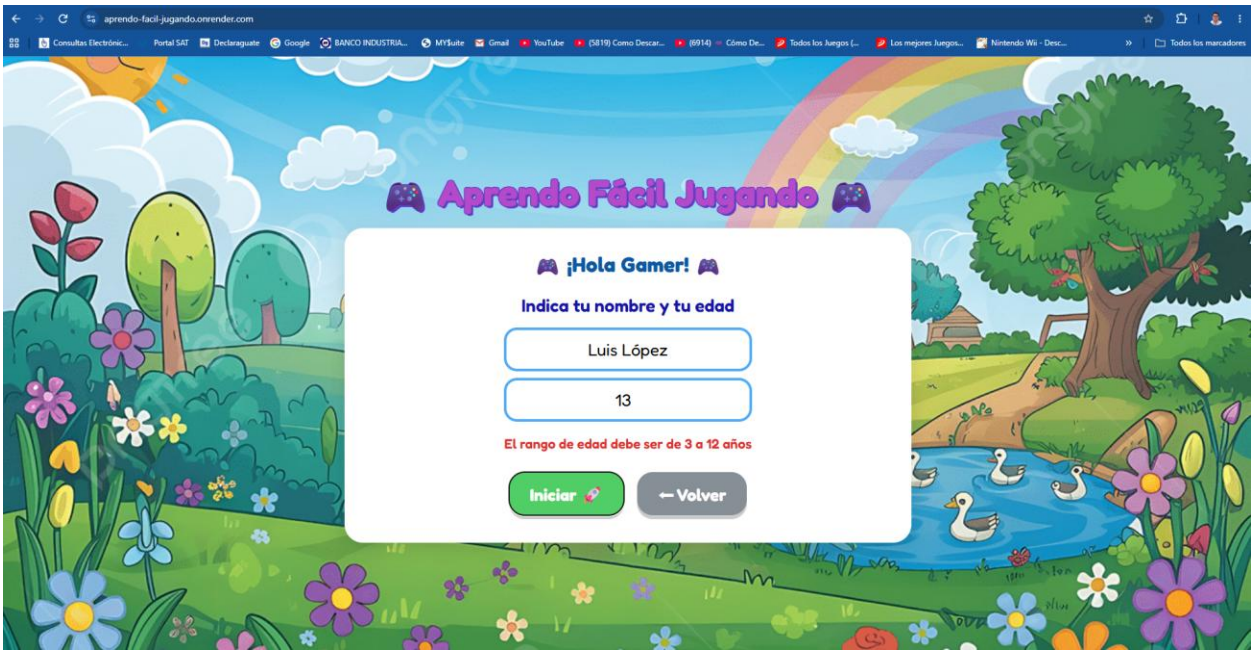


Figura 11. Registro con edad fuera de rango mostrando el mensaje de validación

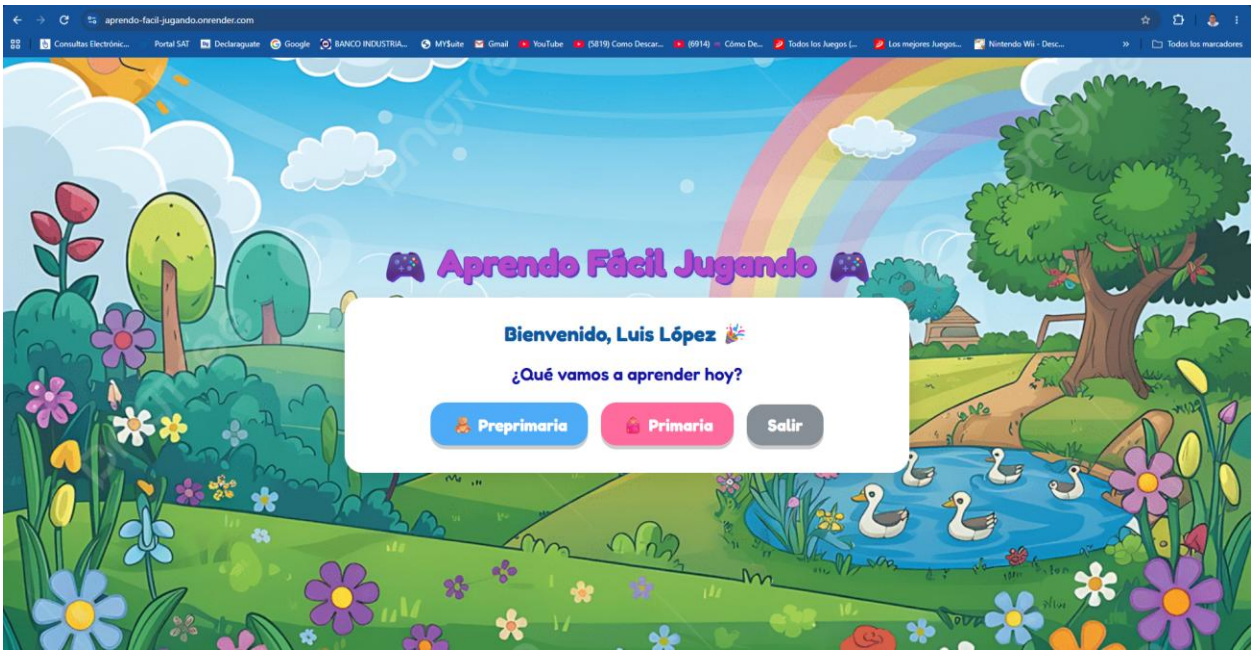


Figura 12. Menú de bienvenida para ingresar a las actividades de Preprimaria, primaria y salir

9.2 Menús de actividades



Figura 13. Menú de actividades de Preprimaria con las 6 actividades y sus imágenes

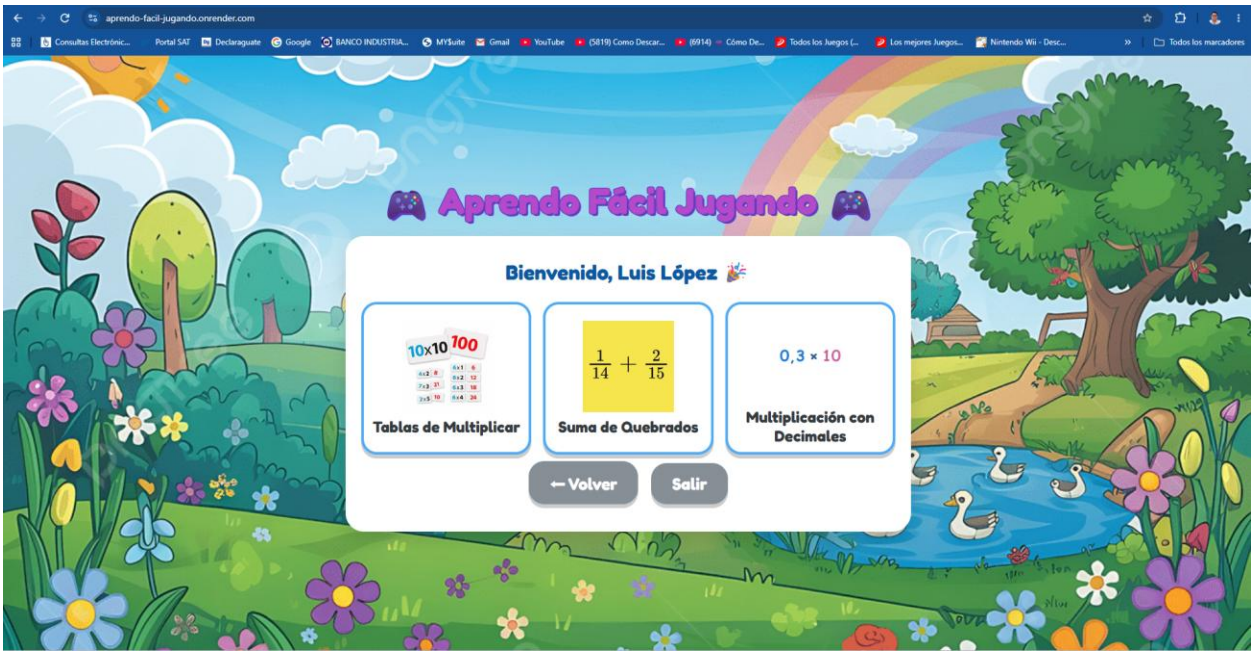


Figura 14. Menú actividades de Primaria con las 3 actividades de matemáticas

9.3 Juegos de preguntas

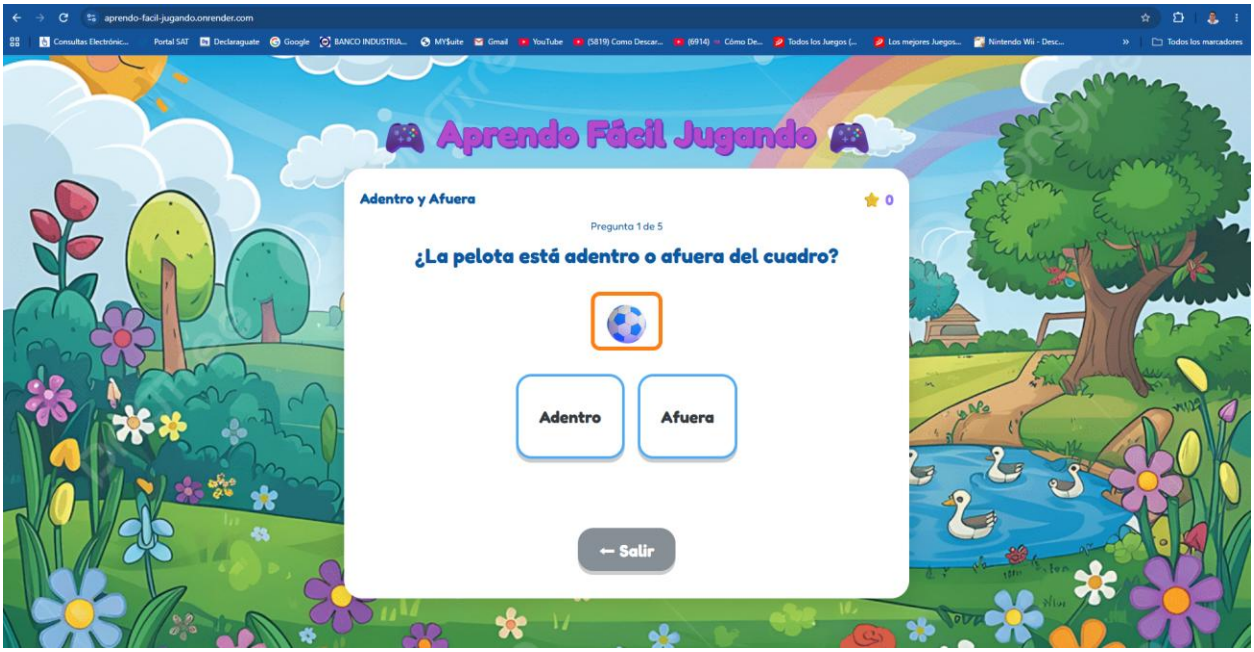


Figura 15. Juego de preprimaria (por ejemplo Adentro y Afuera) con la pregunta y las opciones



Figura 16. Respuesta correcta: la opción se marcan con letras verde y aparece el mensaje Felicidades Bien hecho

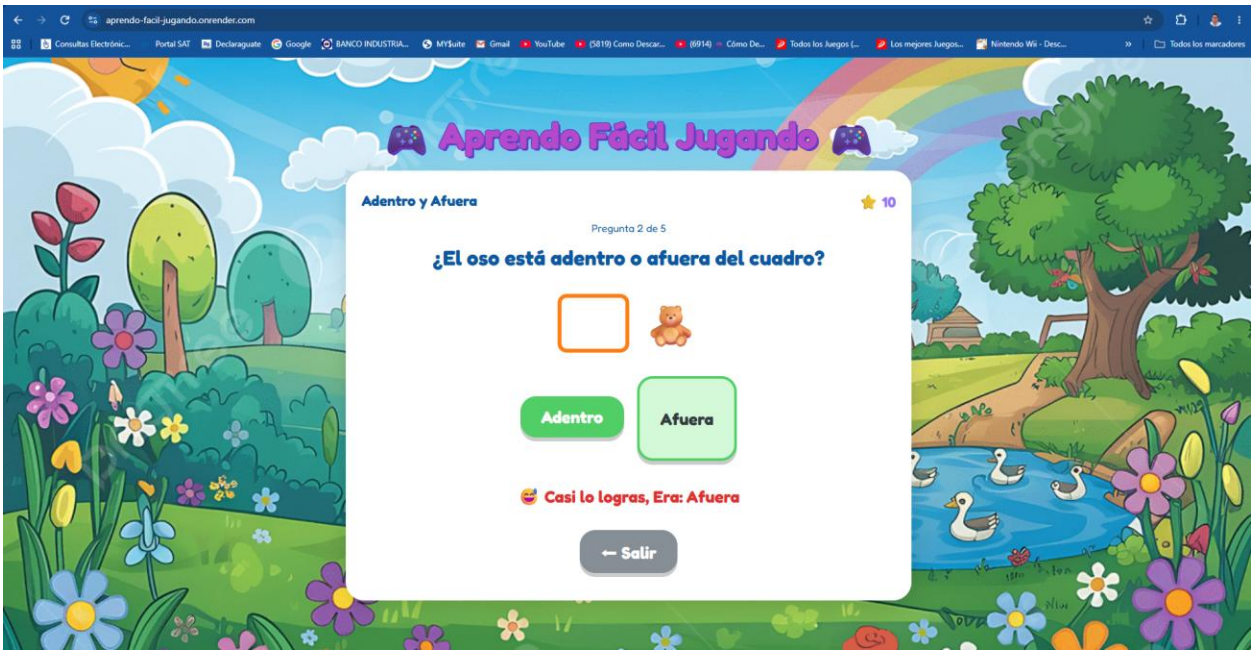


Figura 17. Respuesta incorrecta: la opción se marca con letras rojas Caso lo logras, Era: y muestra la respuesta correcta

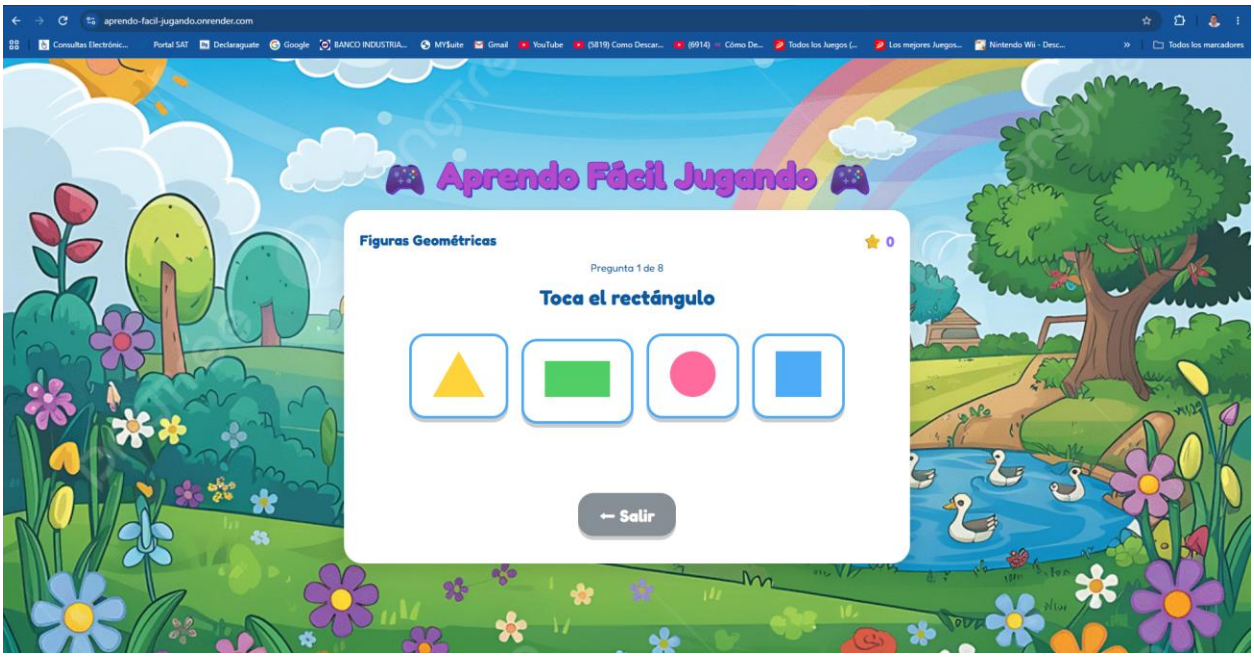


Figura 18. Juego de Figuras Geométricas con las cuatro figuras como opciones

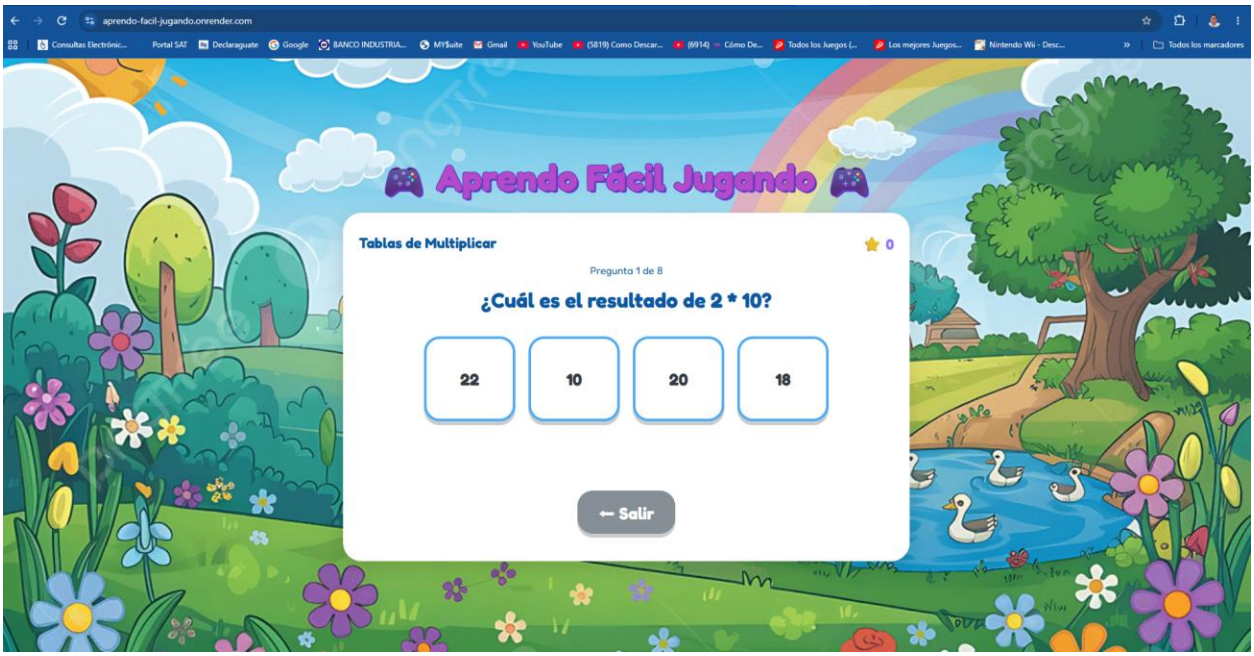


Figura 19. Juego de primaria (tablas, quebrados o decimales) con su operación

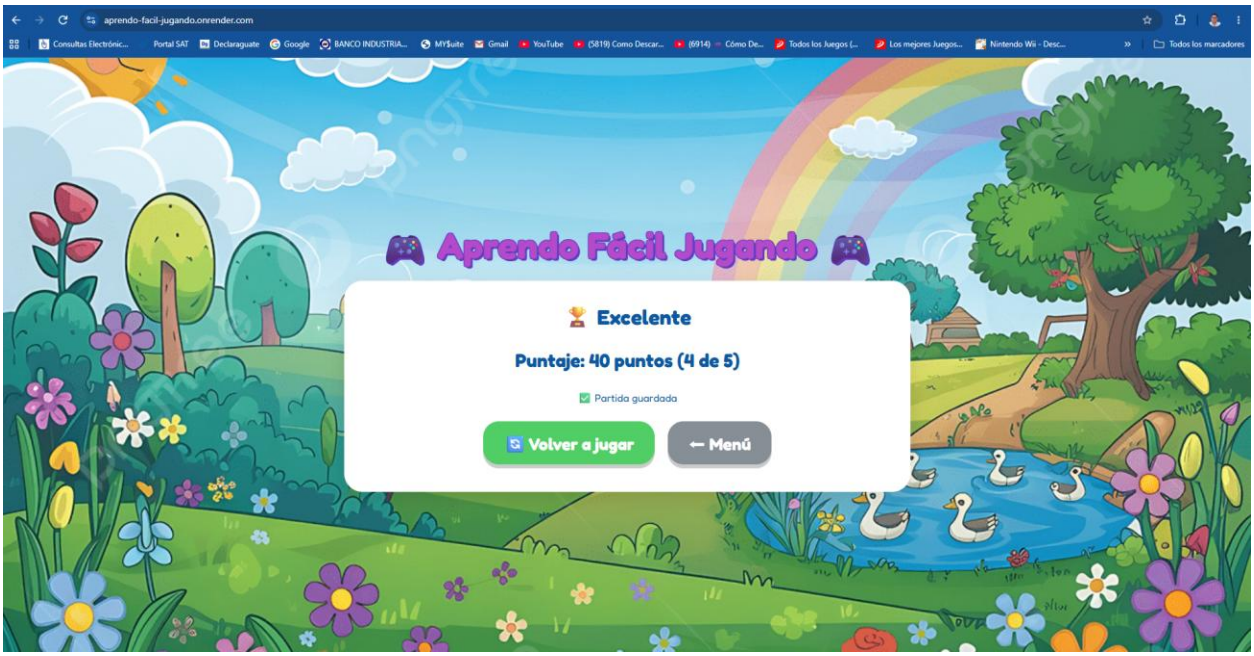


Figura 20. Pantalla de resultados con el puntaje y el mensaje "Partida guardada"

9.4 Juego de memoria

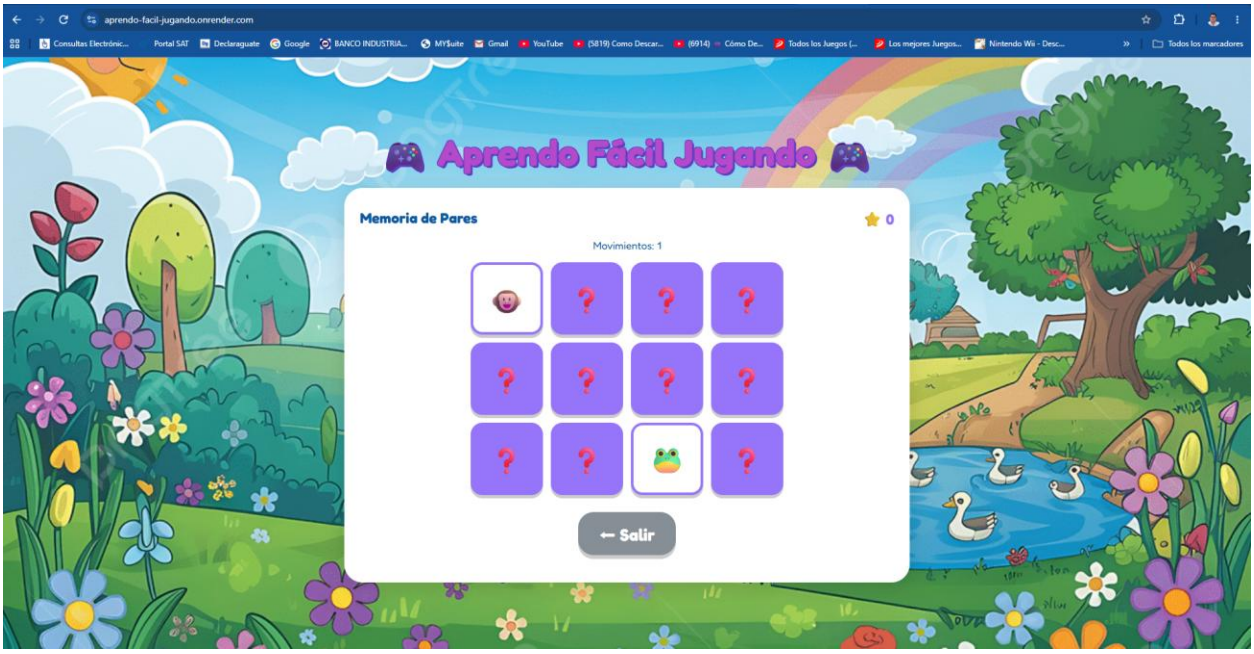


Figura 21. Memoria de pares con algunas cartas volteadas

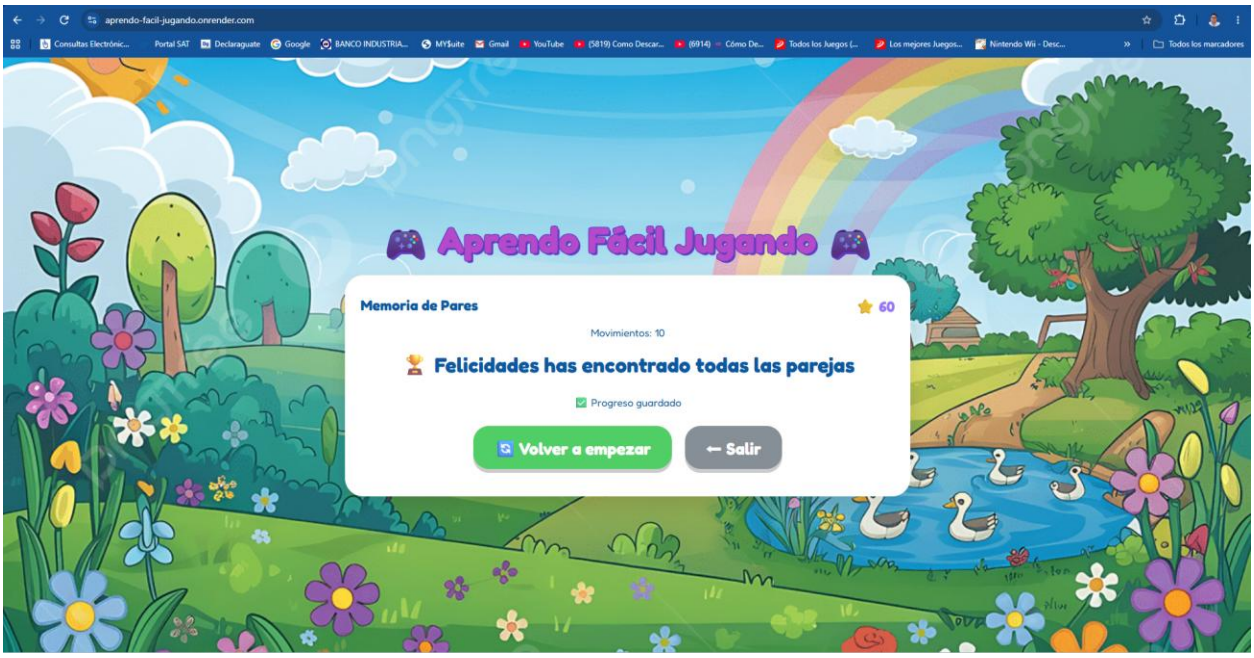


Figura 22. Memoria de pares terminada, con el mensaje final y el progreso guardado

9.5 Tablero del docente

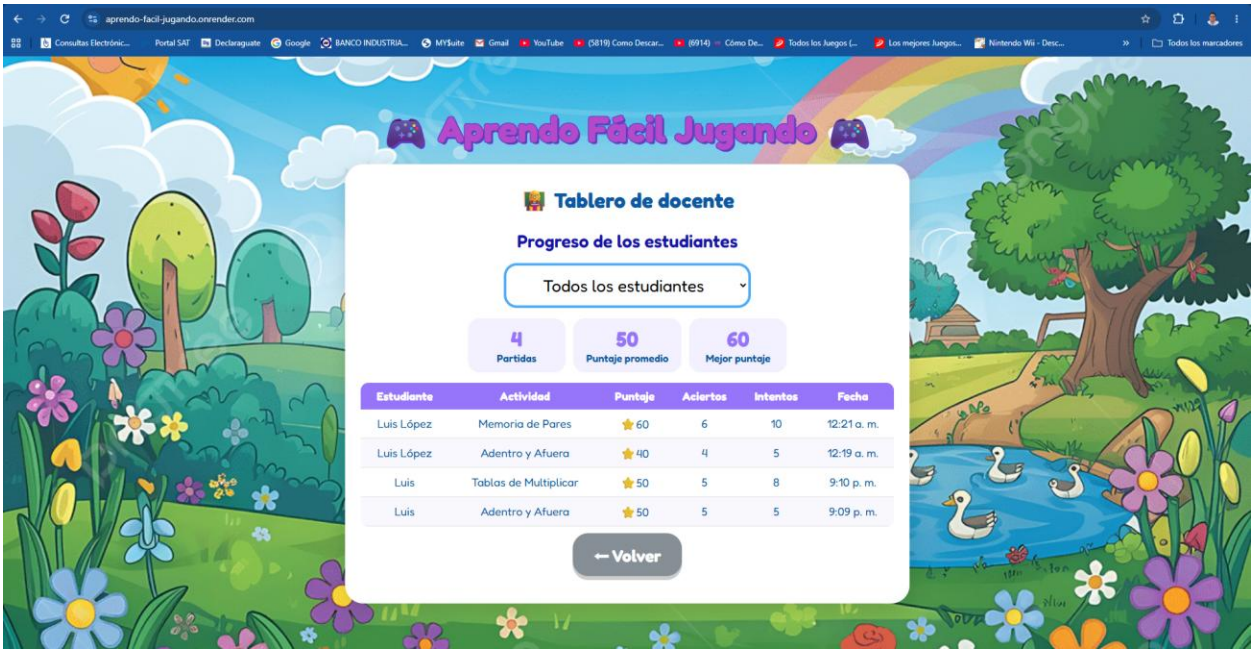


Figura 23. Tablero del docente con las tarjetas de resumen y la tabla de partidas

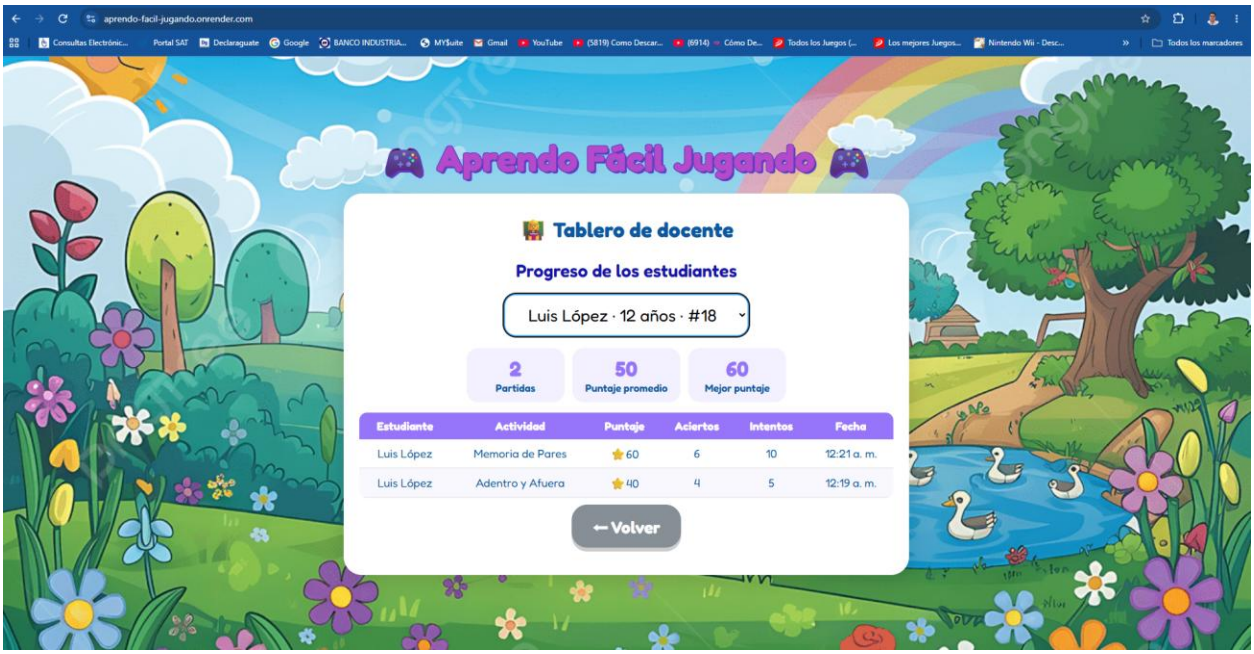


Figura 24. Tablero filtrado por un estudiante seleccionado en la lista

10. Evidencia de los requerimientos funcionales

RF	Qué hice	Dónde está	Evidencia
RF-01	Formulario de nombre y edad que guarda al estudiante en MySQL	Registro.jsx, estudiantes.js, tabla estudiantes	Sección 9.1
RF-02	Juegos con preguntas y opciones distintas en cada partida	Juegos.jsx, Memoria.jsx, data/preguntas.js	Secciones 9.3 y 9.4
RF-03	Verde o rojo y mensaje al instante al responder	Juegos.jsx (responder), App.css	Figuras de 9.3
RF-04	Puntaje que sube 10 puntos por acierto en tiempo real	Juegos.jsx, Memoria.jsx	Marcador ⋆ en 9.3
RF-05	POST que guarda cada partida en MySQL	api.js, progreso.js, tabla progreso	Sección 9.3 y tablero
RF-06	Tablero con el historial por medio de un GET	Tablero.jsx, progreso.js (listar)	Sección 9.5
RF-07	El menú se arma con las actividades que vienen de MySQL	Menu.jsx, actividades.js	Sección 9.2
RF-08	Lista desplegable que filtra el historial por estudiante	Tablero.jsx, progreso.js (WHERE)	Sección 9.5

11. Evidencia de los requerimientos no funcionales

RNF	Qué hice	Dónde está	Evidencia
RNF-01	Fuente redondeada, botones grandes, imágenes y colores para niños	App.css, src/img	Sección 9
RNF-02	Consultas simples con pool de conexiones; medí los tiempos	config/db.js, controllers	Sección 11.1
RNF-03	CORS limitado al origen del frontend	server.js, variable FRONTEND_URL	Sección 11.2
RNF-04	Dos llaves foráneas en la tabla progreso	database.sql	Sección 6
RNF-05	Carpetas backend y frontend independientes	Raíz del proyecto	Sección 5
RNF-06	Probado en Node 24 y MySQL 8; README con la instalación	README.md	Sección 7
RNF-07	Componentes en React; rutas y controladores en Express	components/, routes/, controllers/	Secciones 8 y 9

11.1 Tiempos de respuesta (RNF-02)

Medí los tiempos de dos maneras con la aplicación ya publicada: con la pestaña Network del navegador y con el comando `curl` desde la terminal. Los resultados fueron los siguientes:

Qué hace la petición	Método y ruta	Estado	Tiempo	¿Menor a 500 ms?
Verificación CORS previa al registro	OPTIONS /api/estudiantes	204	201 ms	Sí
Registrar un estudiante	POST /api/estudiantes	201	98 ms	Sí
Catálogo de preprimaria	GET /api/actividades?nivel=preprimaria	304	97 ms	Sí
Catálogo de primaria	GET /api/actividades?nivel=primaria	304	139 ms	Sí
Verificación CORS previa al guardado	OPTIONS /api/progreso	204	186 ms	Sí
Guardar una partida	POST /api/progreso	201	89 ms	Sí
Lista de estudiantes del tablero	GET /api/estudiantes	200	86 ms	Sí
Historial de partidas del tablero	GET /api/progreso	200	343 ms	Sí

Una aclaración importante: el primer intento tardó casi 13 segundos porque el plan gratuito de Render apaga el servidor después de unos 15 minutos sin uso y la primera petición tiene que despertarlo. Una vez activo, todas las consultas respondieron en menos de 500 ms, incluyendo la latencia de internet entre Guatemala y el servidor.

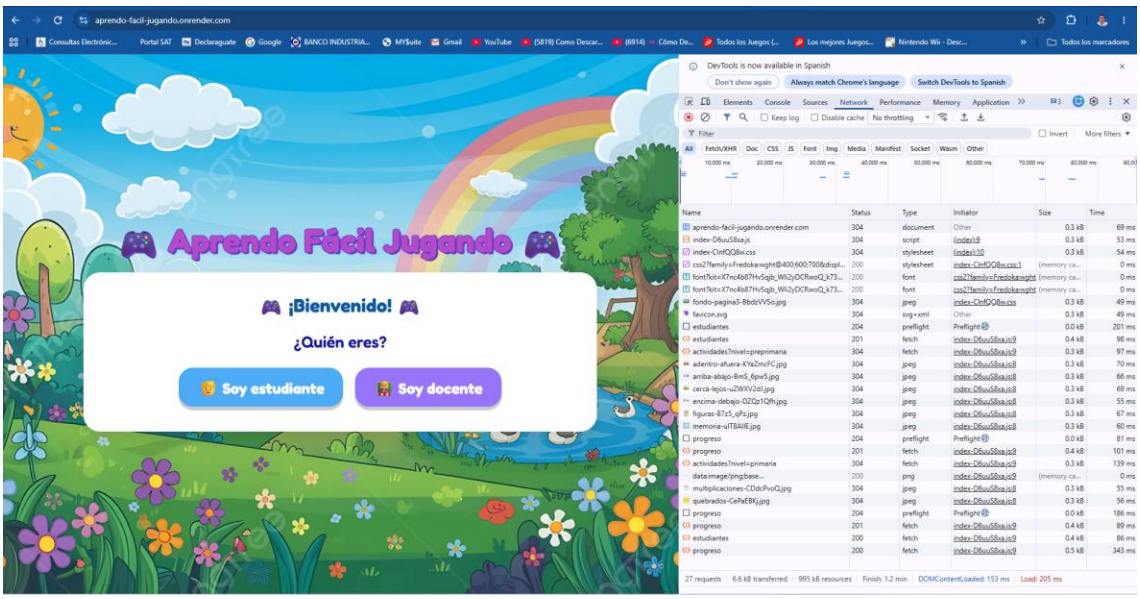


Figura 25. Pestaña Network del navegador con los tiempos de las peticiones a la API (columna Time)

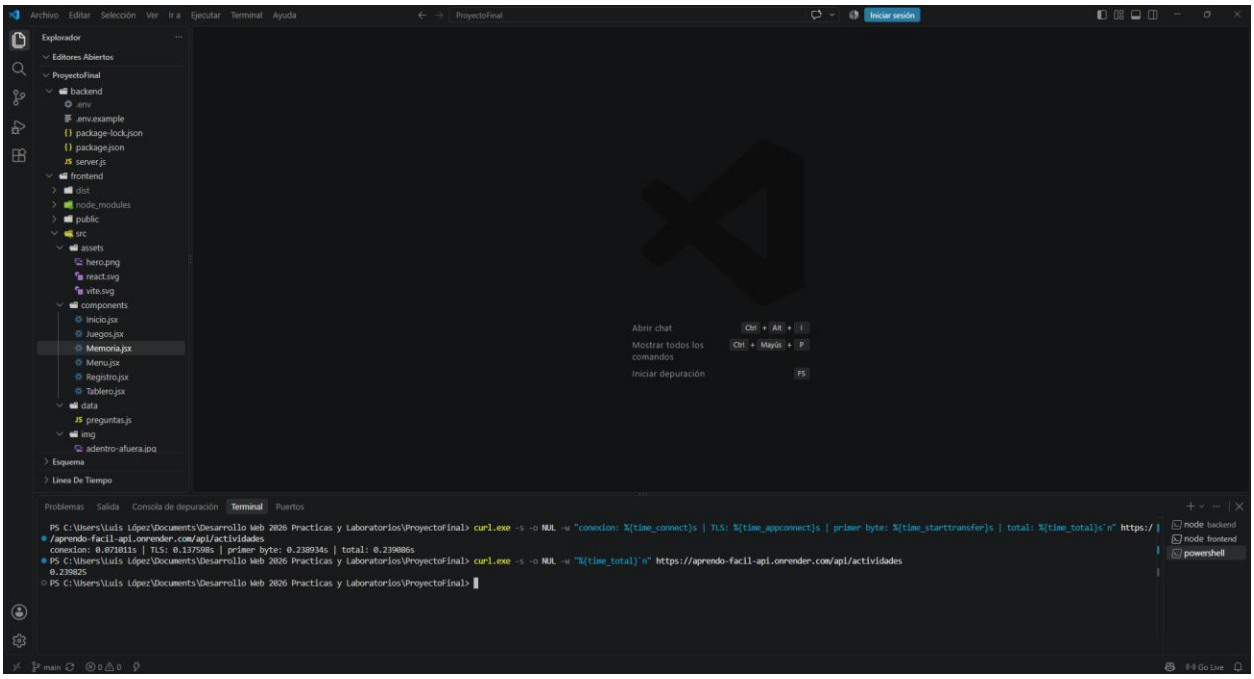


Figura 26. En la terminal los datos aparecen en segundos. Quedan así: Conexión TCP = 71ms, Negación TLS(HTTPS) = 138ms, Espera del Servidor(Consilta MySQL y Respuesta = 239ms

11.2 CORS (RNF-03)

Configuré el backend para que solo acepte peticiones del sitio del frontend. Lo comprobé con `curl` enviando el encabezado Origin, y la respuesta incluyó `access-control-allow-origin` con la dirección de mi sitio publicado.

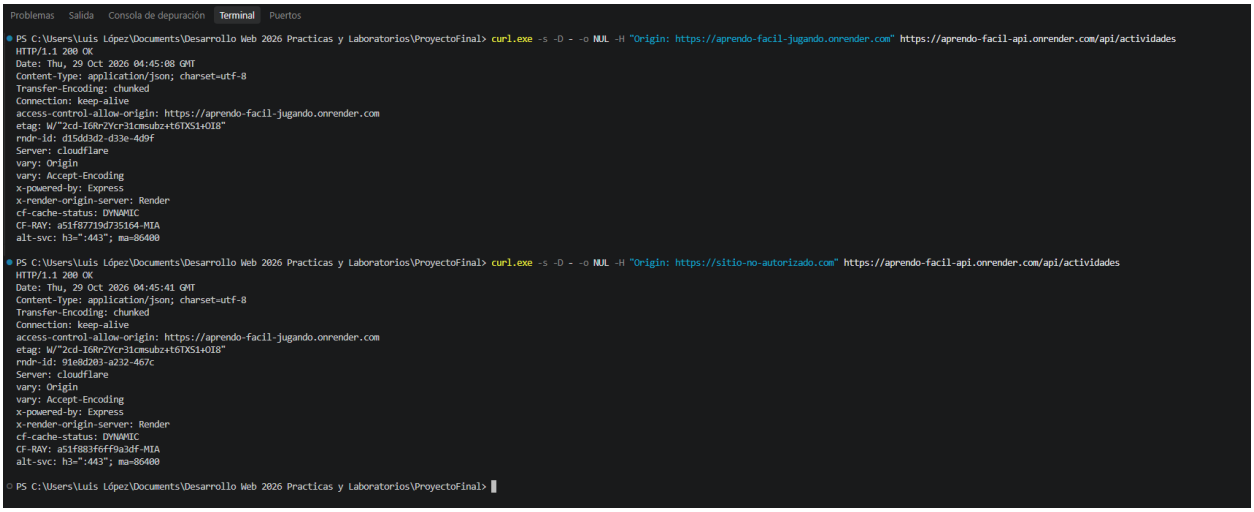


Figura 27. Respuesta de curl mostrando el encabezado access-control-allow-origin

12. Publicación en la nube

Para cumplir con la publicación en un webhosting usé servicios gratuitos. Como no encontré un único hosting gratuito que incluya Node.js y MySQL 8, separé las piezas, algo que además va de acuerdo con el RNF-05 (arquitectura desacoplada):

Pieza	Plataforma	Configuración principal
Base de datos	Aiven (MySQL, plan gratuito)	Servicio aprendo-facil-db; script database.sql cargado desde Workbench con SSL
Backend	Render Web Service (gratis)	Root Directory: backend · Build: npm install · Start: node server.js
Frontend	Render Static Site (gratis)	Root Directory: frontend · Build: npm install && npm run build · Publish: dist
Código	GitHub (repositorio privado)	Render publica automáticamente cada vez que subo cambios a la rama main

Las variables de entorno las configuré en el panel de cada servicio, sin escribirlas en el código. En el backend: `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `DB_SSL` y `FRONTEND_URL`. En el frontend: `VITE_API_URL`, con la dirección de la API.

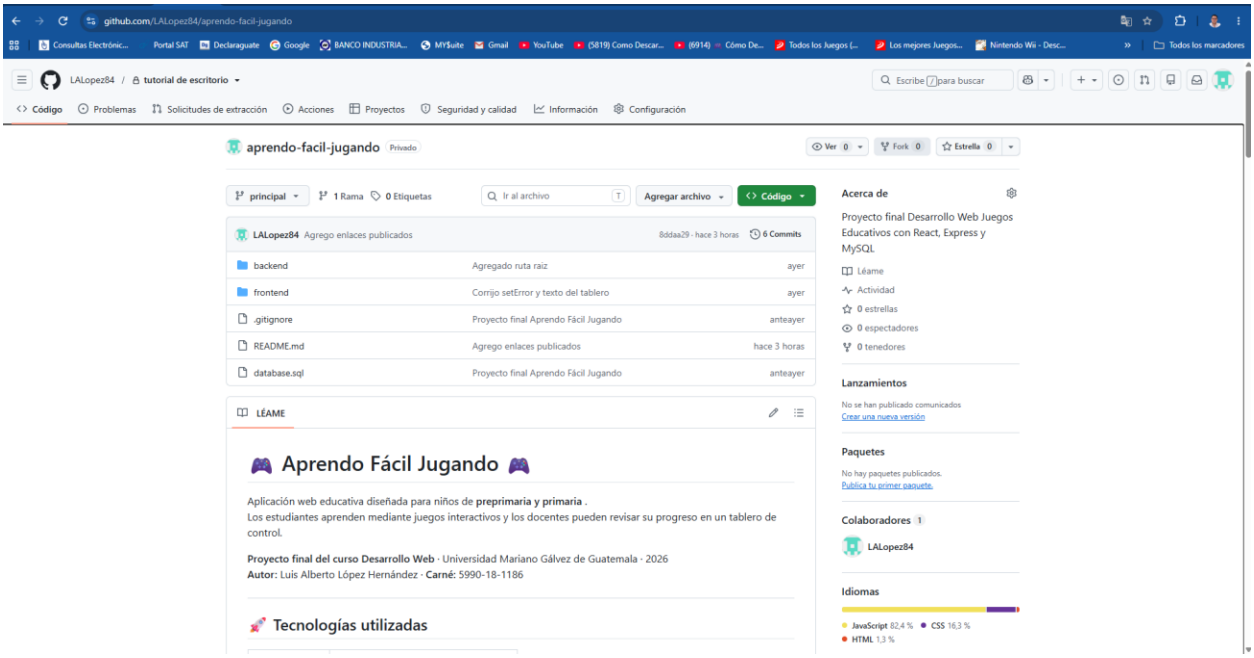


Figura 28. Repositorio en GitHub con las carpetas backend y frontend, .gitignore, el README y database.sql

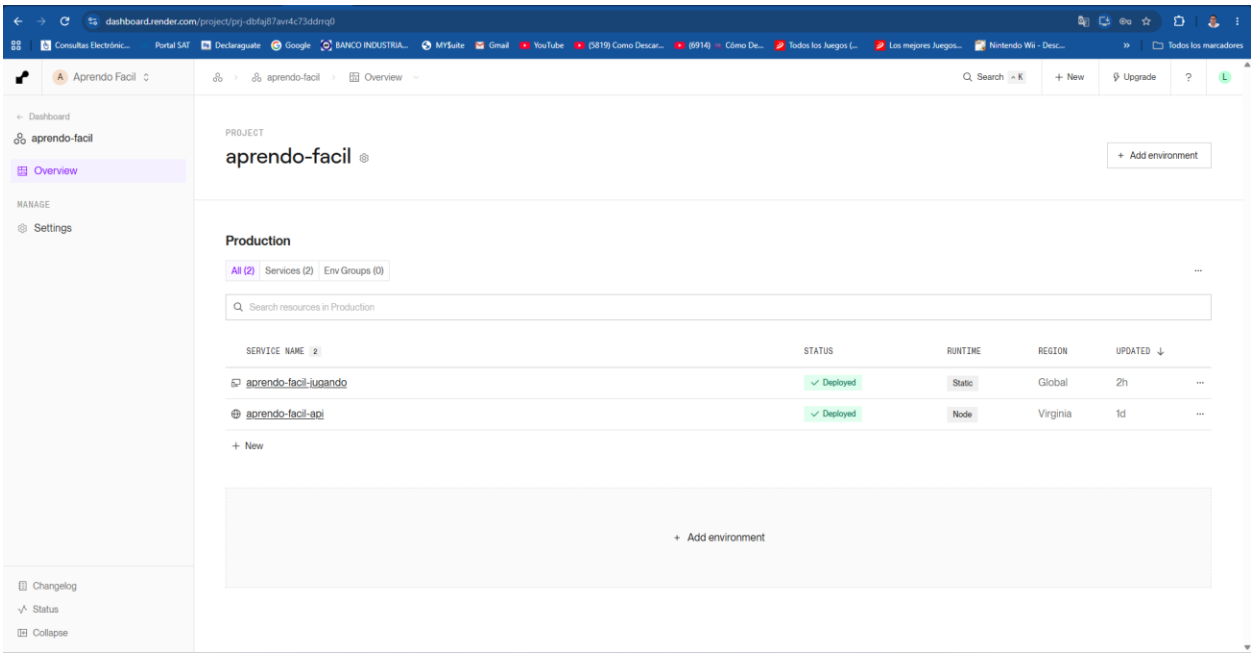


Figura 29. Panel de Render con los dos servicios (Static Site y Web Service) en estado Deployed

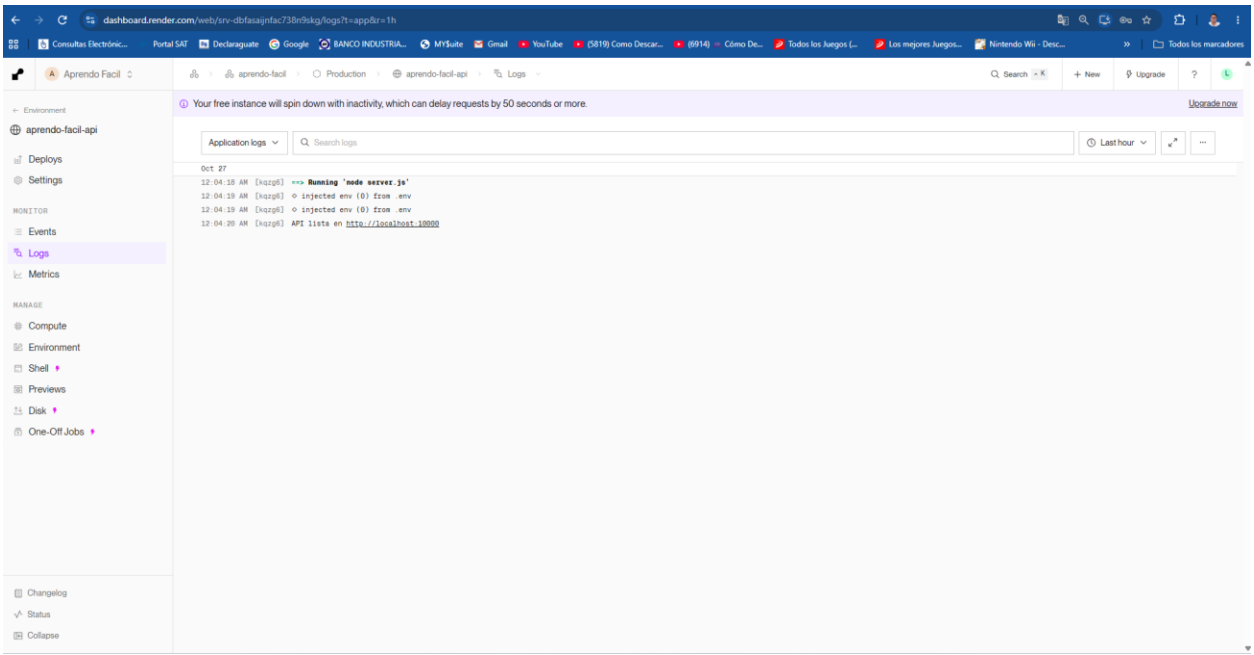


Figura 30. Registro Logs del backend en Render

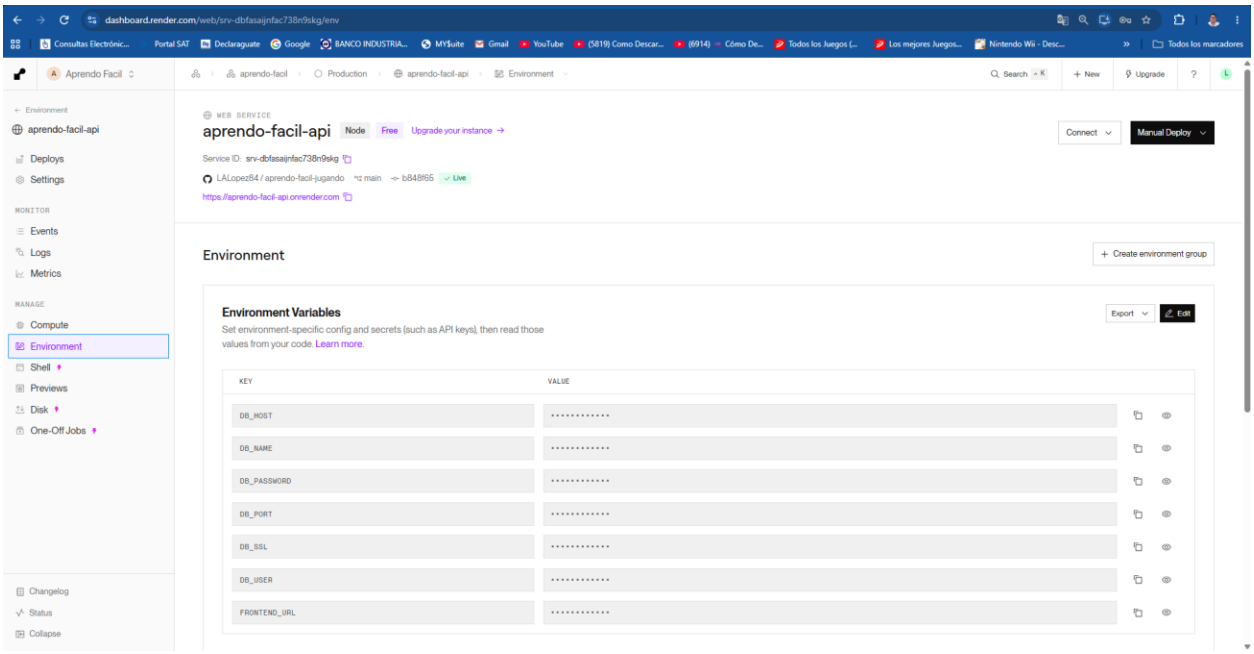


Figura 31. Variables de entorno del backend en Render (valores sensibles ocultos)

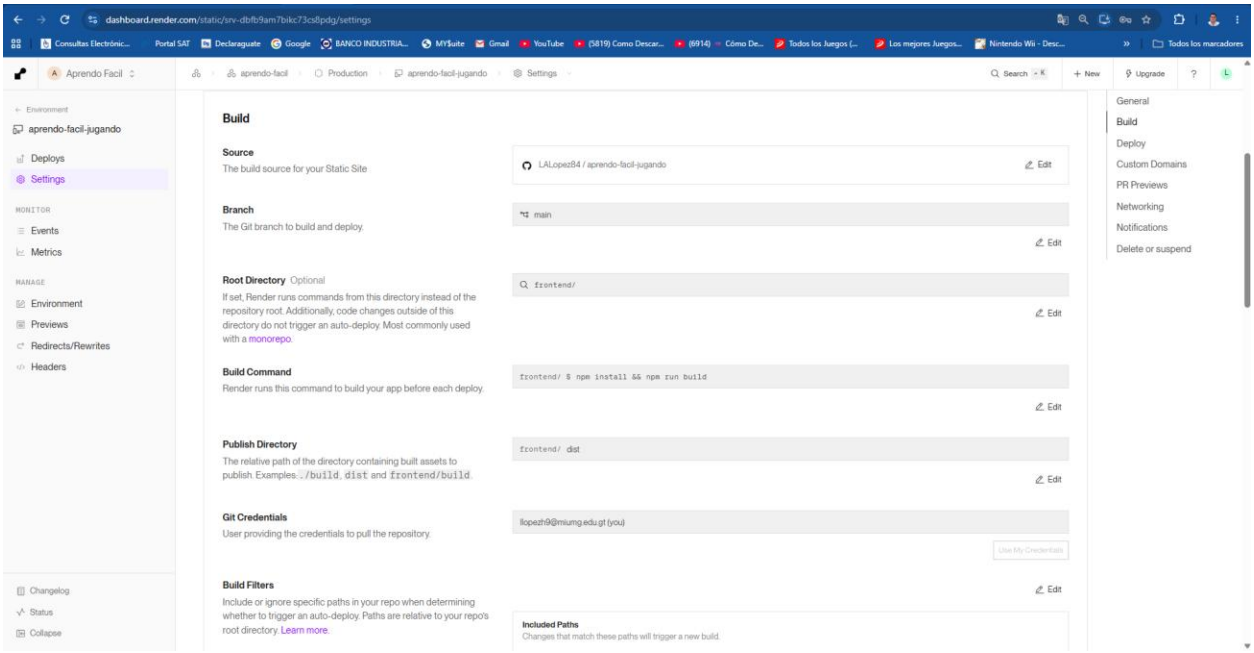


Figura 32. Configuración del Static Site en Render (Root Directory, Build Command y Publish Directory)

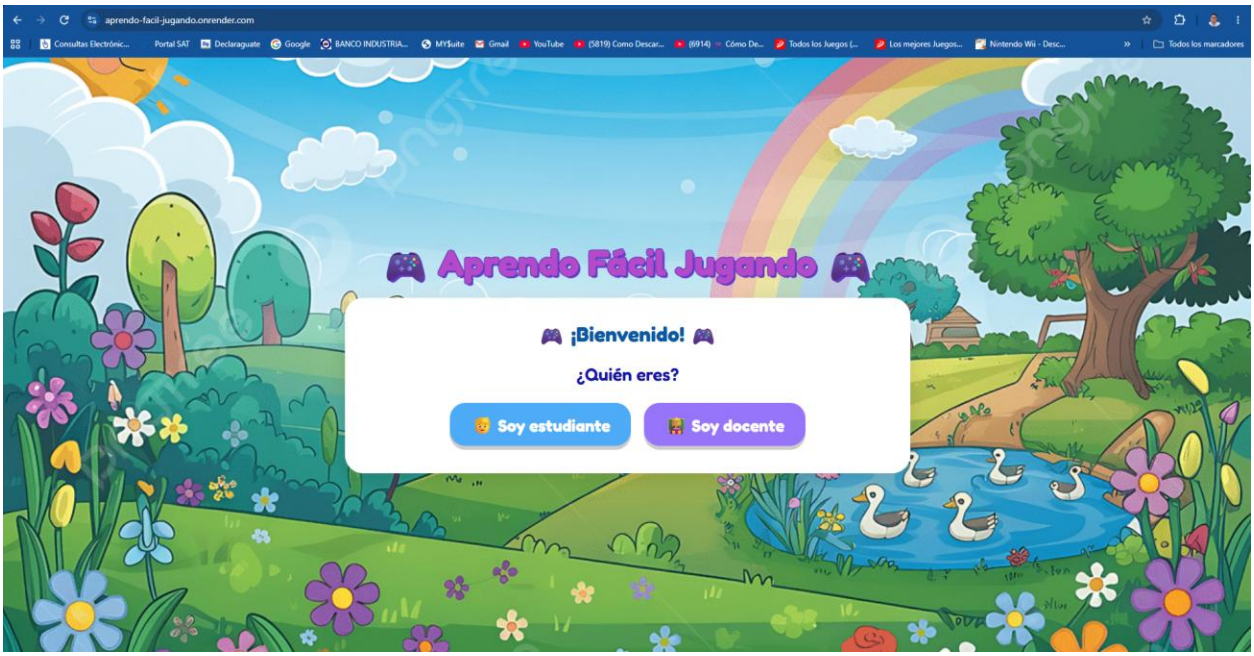


Figura 33. Aplicación publicada funcionando en el navegador, con la dirección pública visible

13. Seguridad aplicada

Apliqué medidas básicas de seguridad que explico con más detalle en el documento técnico:

- Las contraseñas y datos de conexión están en variables de entorno; el archivo `.env` no se sube a GitHub y se entrega solo una plantilla `.env.example`.
- Todas las consultas a MySQL usan parámetros (`?`), lo que evita la inyección SQL.
- Validé los datos de entrada tanto en el frontend como en el backend.
- CORS está restringido a un solo origen, y la conexión a la base de datos en la nube usa SSL.
- Los errores del servidor se devuelven con un mensaje genérico, sin detalles internos.

También reconozco lo que no implementé: la aplicación no tiene inicio de sesión con contraseña (el registro pide solo nombre y edad, como indica el RF-01), el tablero del docente no está protegido y no usé limitación de peticiones. Lo dejé documentado como mejora futura.

14. Dificultades que encontré y cómo las resolví

- **Errores de escritura en nombres.** Varias veces el programa falló porque escribí un nombre distinto al declarado (por ejemplo `serError` en lugar de `setError`, o la tabla `estudiante` en lugar de `estudiantes`). Aprendí a leer el mensaje de la consola del navegador (F12) y de la terminal del backend, que indican el archivo y la línea, y a usar la búsqueda y el reemplazo de VS Code.
- **Errores en las consultas SQL.** Un `WERE` en lugar de `WHERE` y signos `?` sin valores devolvían errores 500. Los resolví revisando el mensaje de MySQL en la terminal.
- **Comunicación entre frontend y backend.** Al publicar, el navegador bloqueó las peticiones por CORS porque el backend solo permitía `localhost`. Lo resolví configurando `FRONTEND_URL` con la dirección real del sitio.
- **Compilación en la nube.** Ejecutar `npm run build` antes de publicar me permitió detectar que faltaba una imagen en el repositorio, algo que habría hecho fallar el despliegue en Render.
- **Conexión a Aiven.** La base en la nube exige SSL, así que agregué la opción `DB_SSL` en la conexión y probé primero desde mi computadora antes de subirlo a Render.
- **Servidor gratuito dormido.** Comprobé que la primera petición tarda unos segundos; por eso abro el backend antes de presentar y lo dejé documentado en el RNF-02.

15. Conclusiones

- Logré construir y publicar una aplicación completa, con interfaz en React, API en Express y datos en MySQL, que cumple los ocho requerimientos funcionales y los siete no funcionales del enunciado.
- Trabajar con frontend y backend separados me ayudó a entender cómo viajan los datos desde un clic del niño hasta una fila en la base de datos, y cómo los errores se rastrean en cada capa.

- Publicar en la nube me enseñó la importancia de las variables de entorno, de CORS y de las diferencias entre trabajar en local y en producción.
- Como mejoras futuras me gustaría agregar inicio de sesión para docentes con contraseñas cifradas, ampliar los bancos de preguntas de los juegos espaciales y más niveles de dificultad, además de pruebas automáticas.

16. Referencias y créditos

Documentación consultada: React (react.dev), Vite (vite.dev), Express (expressjs.com), MySQL (dev.mysql.com), Render (render.com/docs) y Aiven (aiven.io/docs).

Imágenes, fondo y emojis (uso académico):

- https://www.flaticon.com/free-icons/emoji
- https://heroicons.com
- https://github.com/googlefonts/noto-emoji
- https://emojipedia.org/most-popular
- https://es.pinterest.com/ideas/adentro-afuera/914462519444/
- https://ar.pinterest.com/eugiin/arriba-abajo/
- https://www.pinterest.com/jenniferquesadaestrada/concepto-cerca-lejos/
- https://es.pngtree.com/free-backgrounds-photos/elemento-del-paisaje
- https://www.magnific.com/es/vectores/coleccion-comida
- https://math-center.org/es-AR/resources/6th/multiplying-and-dividing-decimals/
- https://impara.es/producto/aprender-a-multiplicar/
- https://soymatematicas.com/sumar-fracciones/